

Phase 1 — libero_spatial_llm Analysis

Short report comparing the LLM-layer-extract KeyBuilder against the prefix-embedding baseline evaluated on `libero_spatial`. Companion figure: `phase1_libero_spatial_llm_results.png`. Baseline figure (libero_spatial only): `../libero_spatial/phase1_results.png`.

1. Setup

1.1 Step A — Extracting the source tensor

Pi0.5’s Stage-1 backbone is a PaliGemma prefix-LM. Given an input sequence composed of three image blocks, the tokenised prompt, and a discretised robot-state block, the model embeds and positional-encodes the tokens into an input matrix

$$\mathbf{E} \in \mathbb{R}^{T \times d}, \quad T \approx 968, \quad d = 2048,$$

and runs 18 decoder blocks. Writing the block operator as $f_\ell(\cdot)$ (self-attention + MLP + residual + RMSNorm) and the running hidden state as $\mathbf{H}_\ell$, we have

$$\mathbf{H}_0 = \mathbf{E}, \quad \mathbf{H}_{\ell+1} = f_\ell(\mathbf{H}_\ell), \quad \ell = 0, \dots, 17.$$

The two experiments differ only in which $\mathbf{H}_\ell$ feeds the key reducer:

Experiment	Source tensor
<code>libero_spatial</code>	$\mathbf{H}_0 = \mathbf{E}$ — the raw pre-LLM input: SigLIP patch embeddings + token-embedding-layer lookups, no self-attention applied
<code>libero_spatial_llm</code>	$\mathbf{H}_{N+1}$ with $N \in \{0, 1, 2, 3\}$ — a shallow hidden state after at most 4 self-attention blocks

The distinction matters: in the baseline each token is still strictly single-modality (a vision patch has never seen the prompt, a prompt token has never seen the image), so the subsequent per-modality reducer + weighted RRF is the *only* place cross-modal fusion can happen. In the LLM-extract variant, one or more self-attention blocks have already fused modalities at the token level before the reducer runs.

Modality masks partition the token axis into disjoint segments $S_{v_0}, S_{v_1}, S_{v_2}, S_\ell, S_{rs}$ (three camera patches, prompt tokens, discretised state). Let $\mathbf{H}^{(f)} = \{\mathbf{h}_t : t \in S_f\}$ denote the rows assigned to field f .

1.2 Step B — Reducers

Every reducer maps $\mathbf{H}^{(f)}$ into a fixed-length vector k_f used as the cache key for field f . Let $|S_f|$ be the segment length and, for the vision fields, $\mathbf{H}^{(f)}$ be reshaped to a 16×16 patch grid $\mathbf{h}_{v,p,q} \in \mathbb{R}^{2048}$.

a — per_modality_mean_pool

$$k_f = \frac{1}{|S_f|} \sum_{t \in S_f} \mathbf{h}_t \in \mathbb{R}^{2048}.$$

c — per_modality_max_pool (element-wise max over the segment)

$$(k_f)_i = \max_{t \in S_f} (\mathbf{h}_t)_i, \quad i = 1, \dots, 2048.$$

b1 — **per_modality_spatial_pool_16** (vision fields only, 4×4 output grid; non-vision fields fall back to mean pool)

$$k_{v,i,j} = \frac{1}{16} \sum_{(p,q) \in B_{i,j}^{(4)}} \mathbf{h}_{v,p,q}, \quad i, j = 1, \dots, 4,$$

flattened to $k_v \in \mathbb{R}^{4 \cdot 4 \cdot 2048}$, i.e. 32 768 per camera.

b2 — **per_modality_spatial_pool_4** (2×2 output grid; the legacy alias **cp1_spatial_pool_64** refers to the same reducer)

$$k_{v,i,j} = \frac{1}{64} \sum_{(p,q) \in B_{i,j}^{(2)}} \mathbf{h}_{v,p,q}, \quad i, j = 1, 2,$$

giving $k_v \in \mathbb{R}^{8192}$ per camera.

e — **prefix_mean_pool** (LLM-only single-field baseline, used with the constant weight $v_0 = 1.0$, $rs = 1.0$)

$$k_{v_0} = \frac{1}{|M|} \sum_{t \in M} \mathbf{h}_t, \quad M = S_{v_0} \cup S_{v_1} \cup S_{v_2} \cup S_\ell,$$

no per-modality split — the whole prefix collapses into one 2048-D vector.

1.3 Downstream

All configurations share an identical downstream stack: the resulting $\{k_f\}$ are fed into **weighted_rrf_knn** with cosine similarity for the vision / prompt fields and an L_2 -to-similarity kernel for **robot_state**; the CP1 checkpoint uses **always_search** + **always_hit**.

1.4 Sweep grid

Axis	libero_spatial	libero_spatial_llm
Reducer	a , b1 , b2 , c + d (CLIP baseline)	a , b1 , b2 , c + e (prefix_mean_pool baseline)
Extract layer	$N = 17$ (final)	$N \in \{0, 1, 2, 3\}$
Field weights	$w_1, \dots, w_8$ (vision / robot_state)	$w_1, \dots, w_8 + p_1, \dots, p_4$ (adds prompt_emb)
Task suite	libero_spatial , 10 tasks $\times$ 10 episodes	same
Total runs	40	196

2. Results

Aggregated over the **a/b1/b2/c** reducers (the **d**/CLIP and **e**/prefix baselines are excluded from the comparison since they are not part of both grids).

2.1 Headline numbers

Metric	libero_spatial (final layer)	libero_spatial_llm (pooled over layers 0–3)
Cells in grid	32	192
Mean success	49.7 %	51.0 %
Max success	64.0 % (b2/w5) · 66.0 % if CLIP d is counted	67.0 % (a/w6 @ L1, b2/w5 @ L3)
Std-dev across cells	0.090	0.123

2.2 Reducer spread — robustness

Mean success for each reducer, averaged over weight configs (and layers for the LLM variant):

Reducer	libero_spatial	libero_spatial_llm
a (mean_pool)	48.9 %	50.0 %
b1 (spatial_16)	52.2 %	51.1 %
b2 (spatial_4)	49.0 %	50.7 %
c (max_pool)	48.5 %	52.0 %
range	3.7 pp	2.1 pp

The gap between the best- and worst-performing reducers **shrinks from 3.7 pp to 2.1 pp** when the keys are drawn from a shallow LLM layer.

2.3 Layer-wise behaviour

Layer N	Mean	Max	SD
0	46.8 %	66.0 %	0.176
1	52.6 %	67.0 %	0.091
2	52.5 %	66.0 %	0.101
3	52.0 %	67.0 %	0.099

Layer 0 (one self-attention block only) is noticeably worse and more dispersed — not enough cross-modal mixing has happened yet. Layers 1–3 are indistinguishable; the benefit of LLM extraction saturates after a single full mixing block.

2.4 prefix_mean_pool baseline

A single-vector key (no per-modality split, no weight tuning) reaches **58 % at layer 1**, beating the best **libero_spatial** reducer mean. Strong evidence that, once the prefix-LM has mixed modalities, a hand-tuned multi-field weighted RRF is no longer necessary to recover most of the performance.

3. Conclusions

1. **No breakthrough on the ceiling.** Peak success is essentially flat (67 % vs. 66 %); pulling keys from a shallow LLM layer does *not* unlock a new frontier on **libero_spatial**.
2. **But markedly more robust.** The spread across reducers shrinks (3.7 pp → 2.1 pp) and the mean improves by +1.3 pp. The choice of **mean** / **max** / **spatial** pool matters less once modalities are already fused inside the LLM.
3. **One attention block is enough.** Layer 0 is too shallow; layers 1–3 are statistically interchangeable.

4. **The bottleneck is no longer the key builder.** Two very different families (end-of-prefix pool vs. shallow-LLM pool) converge to the same ~67 % ceiling; remaining headroom has to come from somewhere else (gate/judge policy, search strategy, cache coverage, or action-side changes), not from a smarter reducer.
5. **Expected gain is generalisation, not in-distribution accuracy.** Shallow-LLM keys carry cross-modal (vision + language + state) fusion for free, which should help on tasks that share semantics but not pixels. This benchmark cannot measure that — **validation requires real-robot or cross-task trials.**